OFFICE HOURS · LIVE BUILD · 60 MIN

Agent to Agent Communication

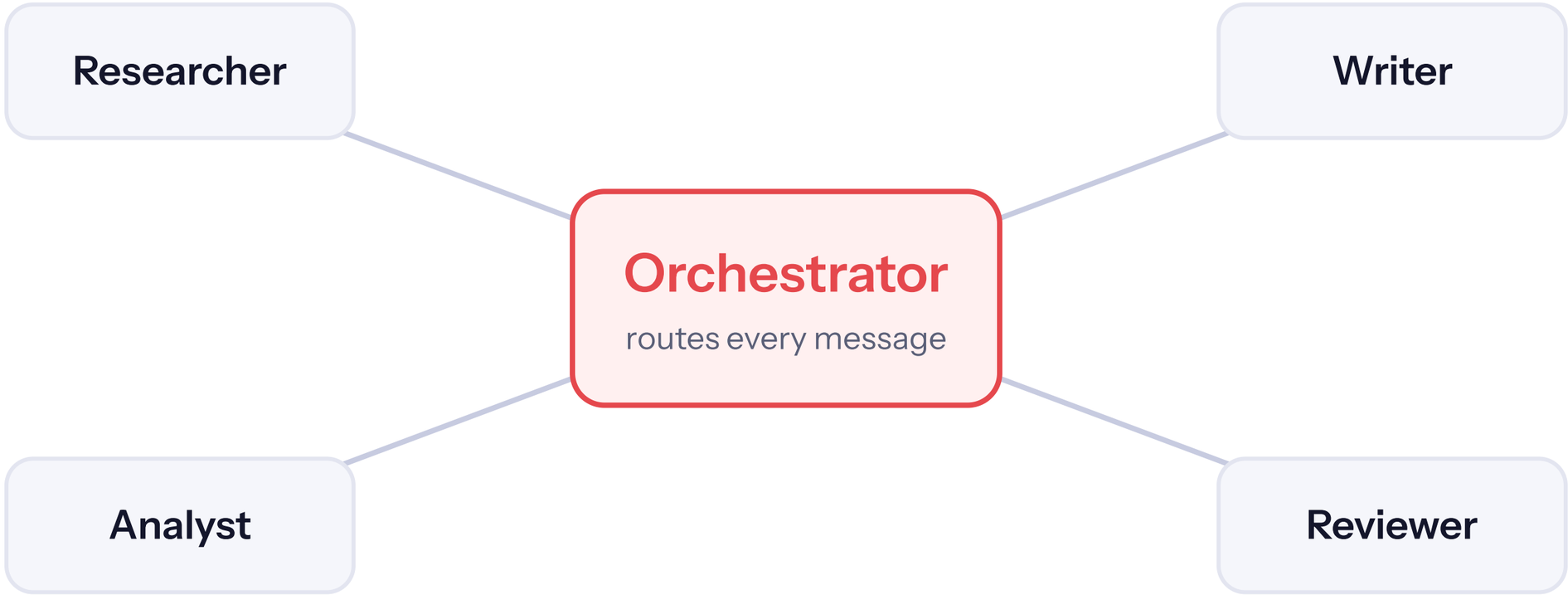
Peer to peer, live.

Ruth Mutua · DevOps Engineer

`$ built live in the terminal`

HOW MOST MULTI-AGENT DEMOS LOOK

Everyone talks through the boss.



So what happens when the boss dies?

WHY AGENTS TALK AT ALL

No single agent is good at everything.

So they delegate, ask, and hand off, the same way a team does.



The real question for today: how do they find each other, and what carries the message?

THE LANDSCAPE

Four ways agents talk today.

Orchestrator

One boss knows every agent and routes every message.

`LangGraph supervisors, most demos`

Message broker

Agents publish and subscribe through a shared pipe.

`Kafka, RabbitMQ, Redis pub/sub`

Protocols

Agreed formats so agents and tools from anyone can connect.

`MCP, A2A`

Peer mesh

Agents find each other by gossip and message each other directly.

`SWIM + ZeroMQ, what we build today`

Each one answers two questions differently: who knows where everyone is, and what dies with it.

CLEARING UP THE MIX-UP

MCP and A2A solve different problems.

MCP

agent ↔ tools and data

How one agent plugs into databases, APIs and files. It's the USB port, not the conversation.

- From Anthropic, November 2024
- An agent calling a tool is not agents talking

A2A

agent ↔ agent, across vendors

How agents built by different teams or companies discover each other and hand over work.

- From Google, April 2025; Linux Foundation since June 2025
- Agents publish an Agent Card so others can find them
- IBM's ACP merged into it in August 2025

Heads up: "ACP" today often means Zed's Agent Client Protocol, which connects coding agents to editors. A different thing again.

SIDE BY SIDE

What breaks when one node dies?

	FINDING EACH OTHER	MESSAGE TRAVELS THROUGH	WHEN ONE NODE DIES	YOU ALSO RUN	BEST FOR
Orchestrator	It holds the list	The orchestrator	**Lose the boss, lose everything**	The orchestrator	Simple pipelines
Broker	Topics and queues	The broker	**Lose the broker, talk stops**	A broker cluster	High-volume events
A2A	Agent Card at a known URL	HTTP between agents	Calls to that agent fail	An endpoint per agent	Agents across companies
Peer mesh	Gossip, via a seed	Direct, peer to peer	**The rest keep talking**	Just the agents	Your own fleet

THE HONEST FRAMING

A2A and a peer mesh aren't rivals.

A2A

`a shared language`

So your agent can work with an agent another company built.

Peer mesh

`a shared neighbourhood`

So your own agents find each other and keep working when one of them dies.

Use the mesh inside your fleet, and speak A2A at the edge.

HOW A MESH WORKS, IN 60 SECONDS

No list. Just neighbours who gossip.



1. knocks on the seed

2. peers ping each other and share news

3. gone quiet: suspect, then dead

Messages go straight from one peer to another. No one in the middle.

WHAT I'LL BUILD TODAY

Three checkpoints, live.

1

Two agents talking

One file, two agents, a direct request and reply.

2

A mesh across terminals

Three terminals, one seed. Kill one and watch the rest keep talking.

3

A new agent, found live

A new specialist joins, and the others find it with no redeploy.

THE SETUP

Everything I'm running.

```
# Python 3.10 or newer. No LLM key, no Redis.
pip install "jarviscore-framework[p2p]"

# terminal 1: the seed
python mesh_node.py analyst 7946
# terminal 2: join through the seed
python mesh_node.py writer 7947 127.0.0.1:7946

# restarted a node? give it a new port
```

- **Try it after the session. The files are in the Discord thread.**

TAKING IT FURTHER

From one laptop to a fleet.

Same code, more machines

Change the bind address and open the ports, TCP and UDP. Nothing else.

Add Redis for durable work

Gossip finds the team. Redis keeps workflows alive through restarts.

Speak A2A at the edge

The mesh runs your fleet. A2A connects you to agents other companies build.

Built for change

Agents come and go freely. The roster catches up on its own, in seconds.

`github.com/Prescott-Data/jarviscore-framework` · `pip install jarviscore-framework` · Questions